

Segment Routing

Programmable Networks A.Y. 2024/25

Outline

- Introduction to Segment Routing
- Segment Routing Policy
- SRv6 and Segment Routing extension Header
- Network Programming

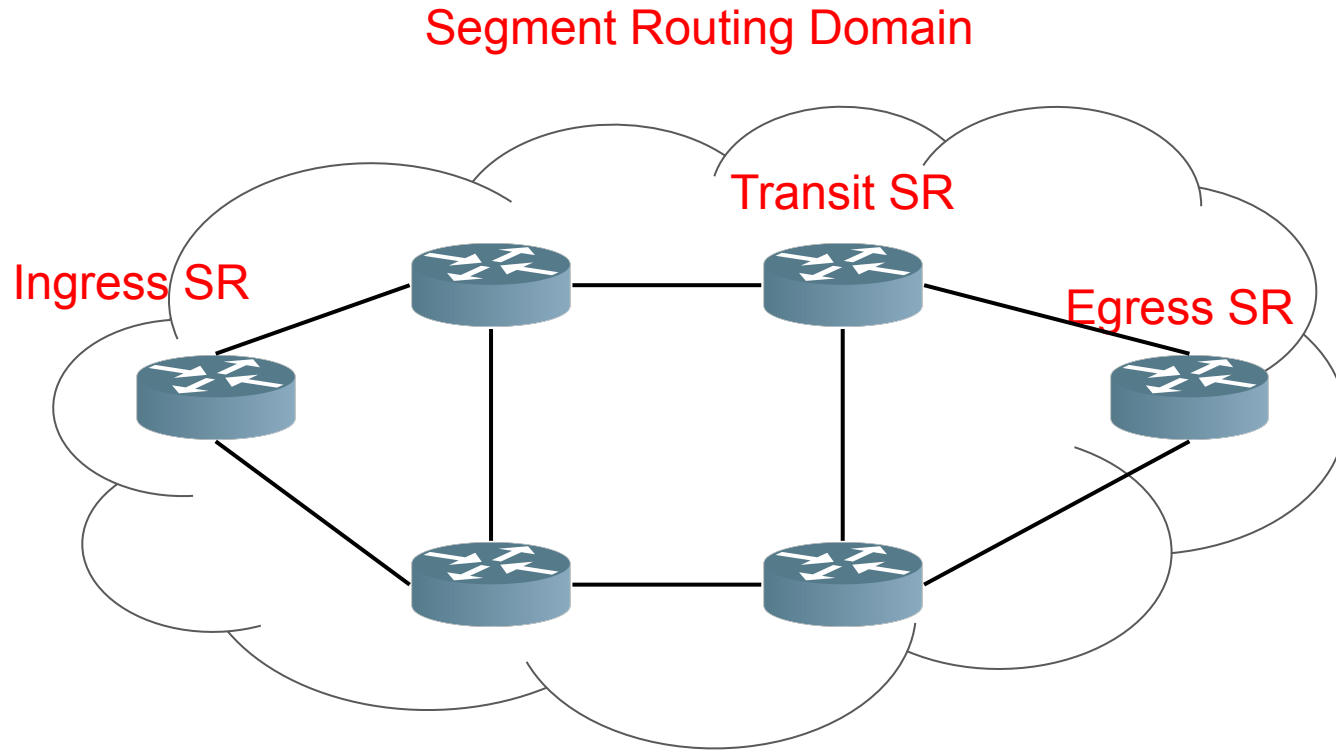
Outline

- Introduction to Segment Routing
- Segment Routing Policy
- SRv6 and Segment Routing extension Header
- Network Programming

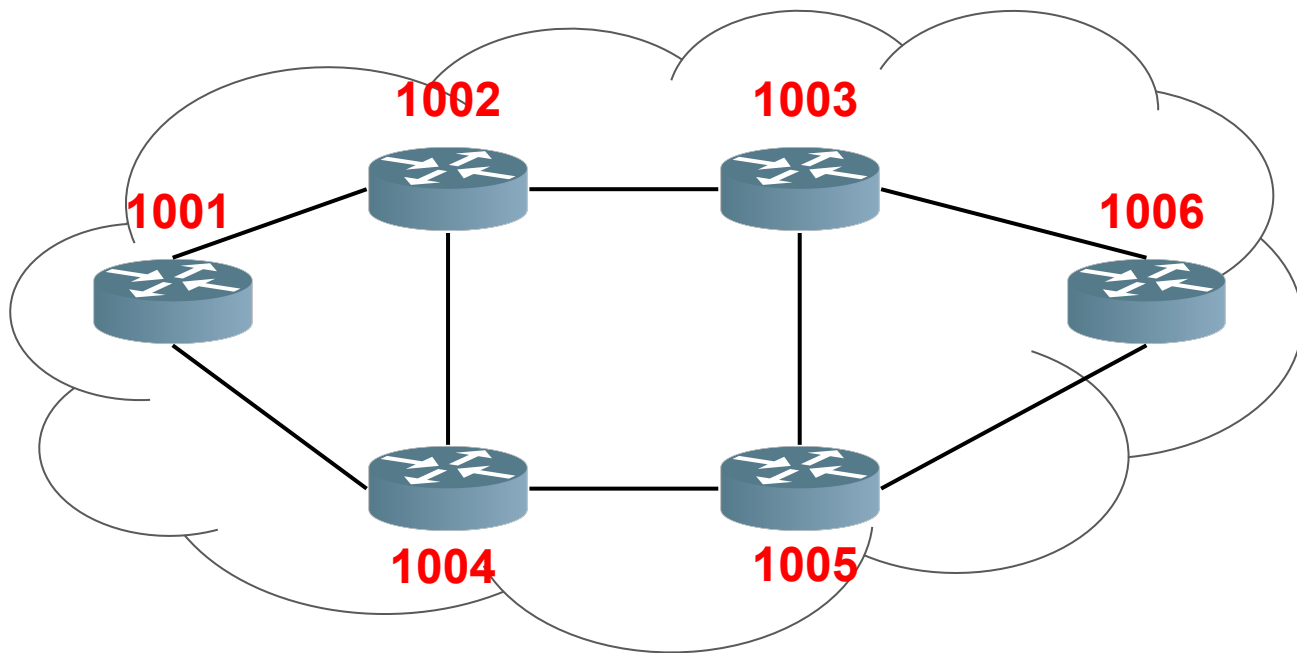
SR overview

- Segment Routing (SR) leverages the **source routing paradigm**
- A node steers a packet through an ordered list of instructions, called **segments**
- A segment can represent any instruction, topological or service-based
- A segment can have a semantic:
 - local to an SR node
 - global within an SR domain
- SR allows to enforce a flow through any topological path while maintaining **per-flow state only at the ingress nodes to the SR domain**

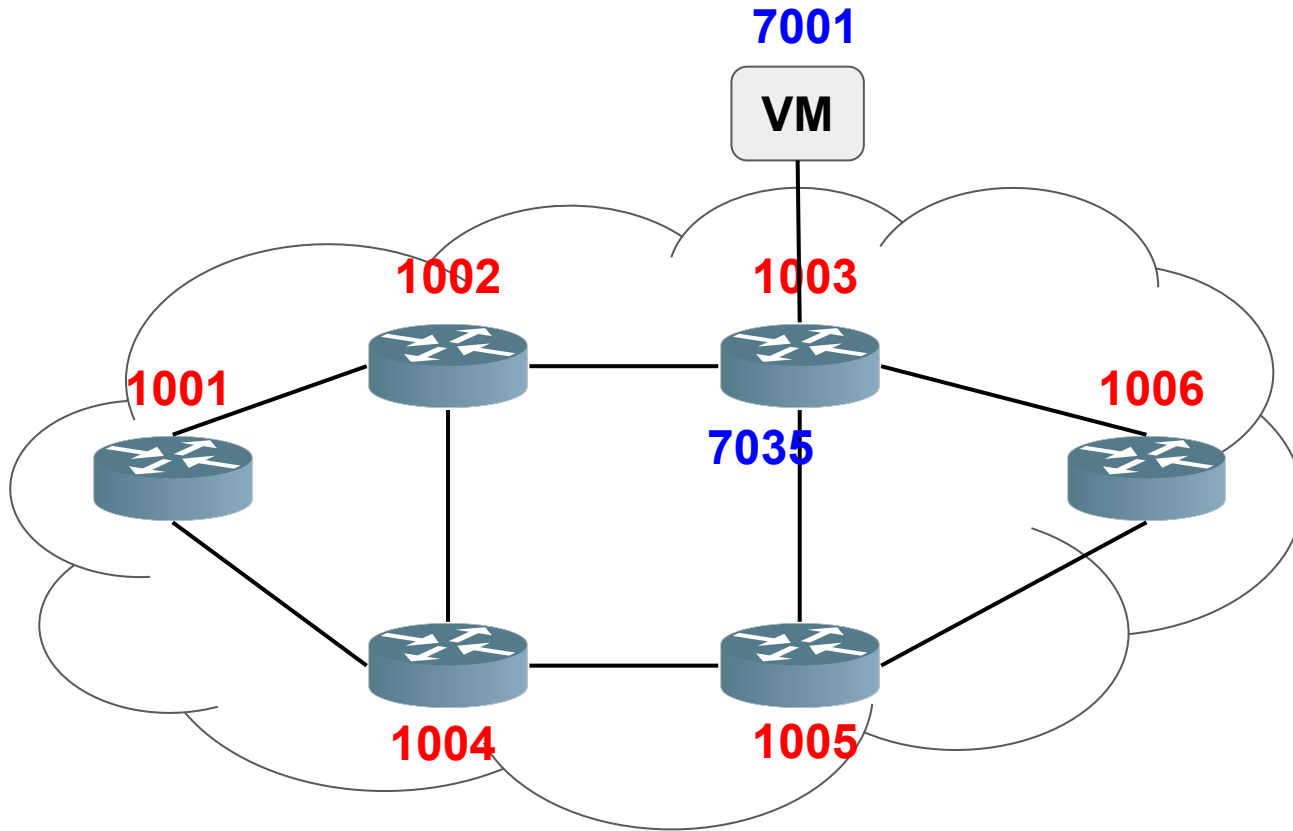
SR terminology



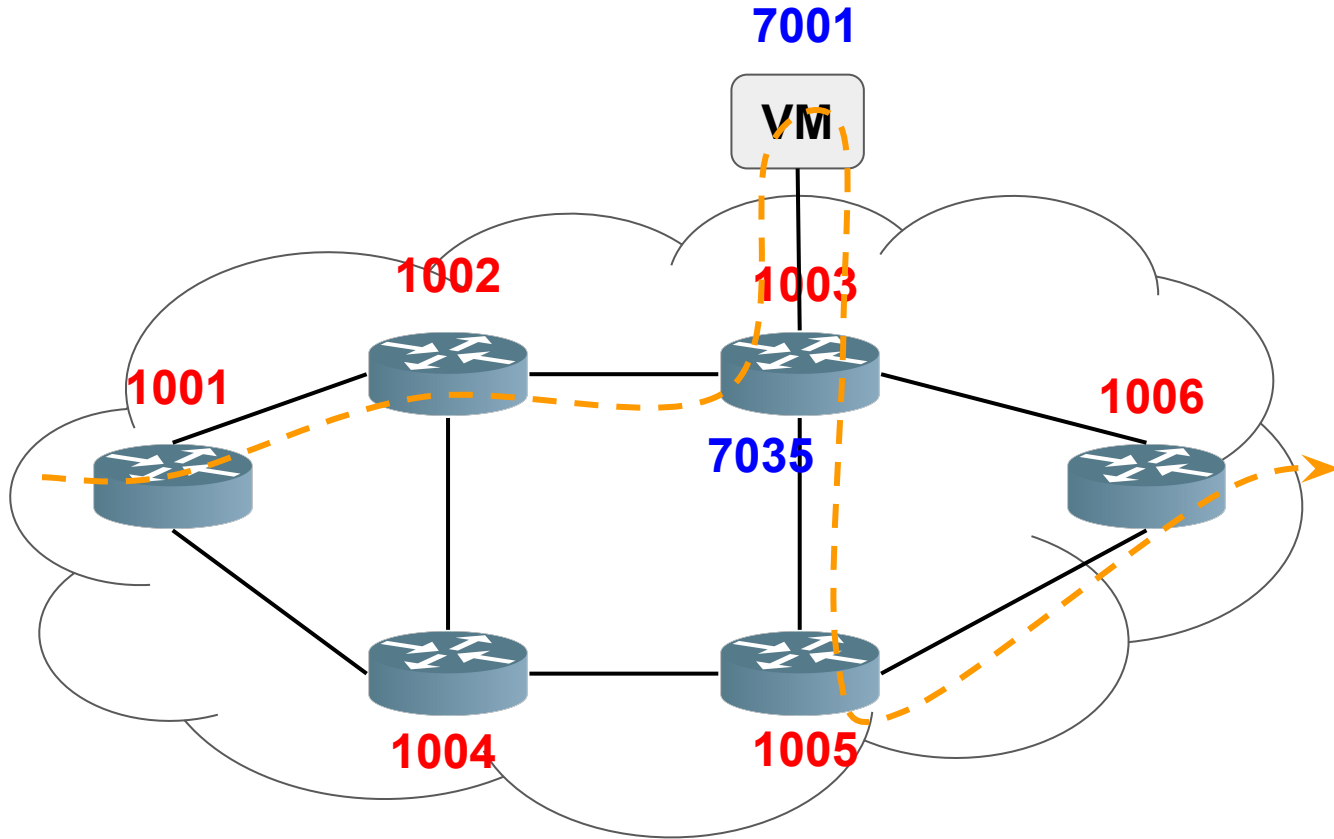
SR idea



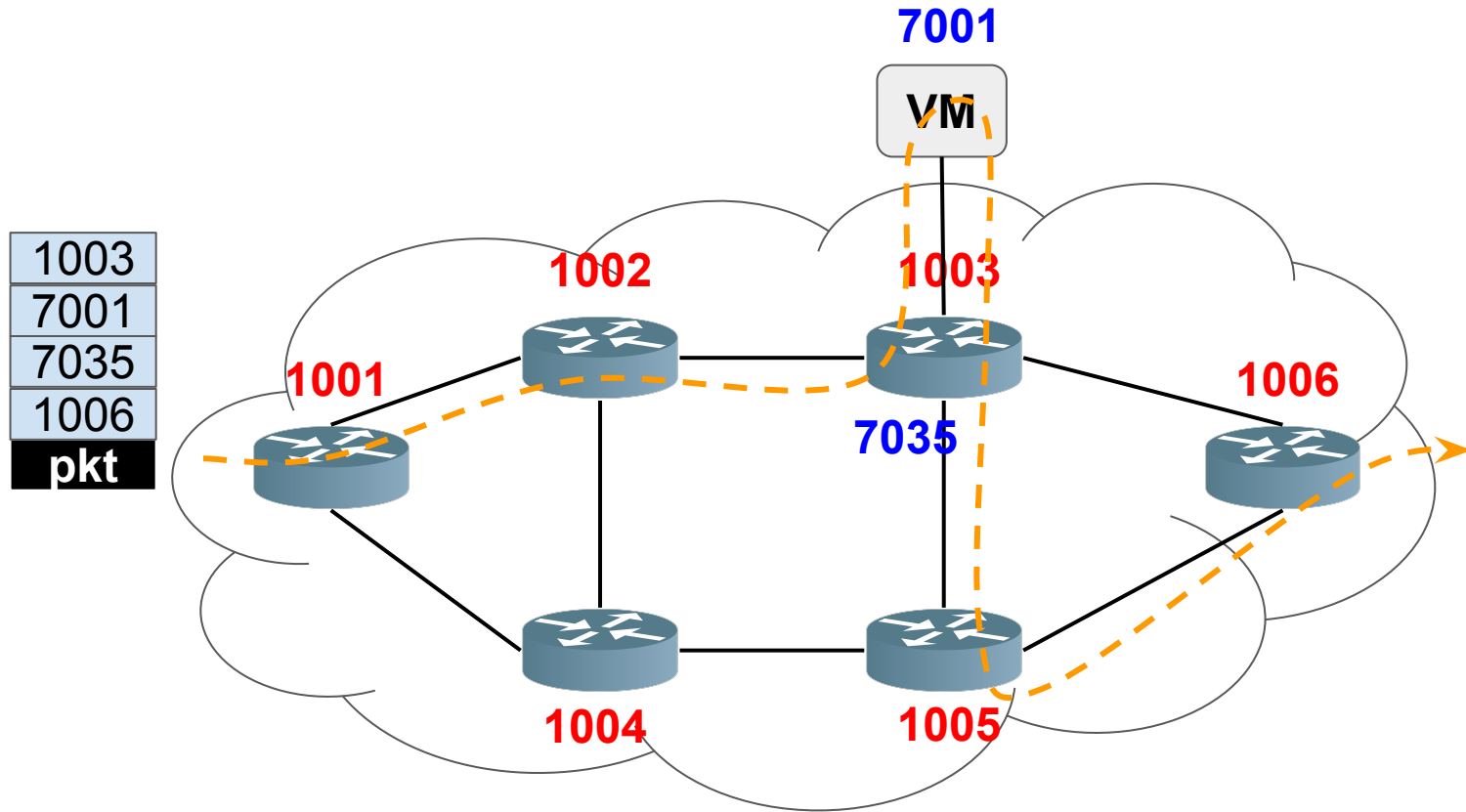
SR idea



SR idea



SR idea



SR control plane

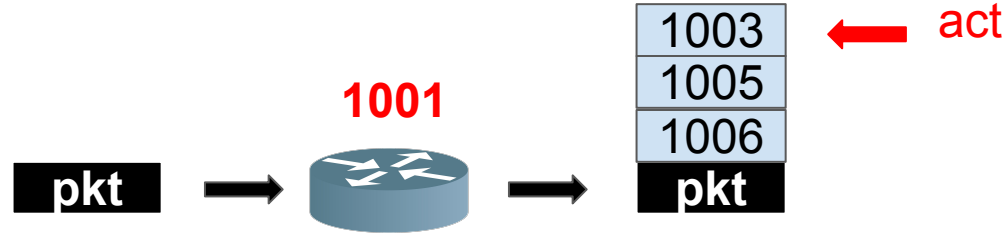
- The SR architecture supports any type of control-plane: distributed, centralized or hybrid.
- In a **distributed scenario**:
 - the segments are allocated and signaled by OSPF or BGP
 - a node individually decides to steer packets on a source-routed policy
 - a node individually computes the source-routed policy
- In a **centralized scenario**:
 - the segments are allocated and instantiated by an SR controller
 - the SR controller decides which nodes need to steer which packets on which source-routed policies
 - the SR controller computes the source-routed policies
- A **hybrid scenario** complements a base distributed control-plane with a centralized controller
 - for example, when the destination is outside the IGP domain, the SR controller may compute a source-routed policy on behalf of an IGP node

SR data plane

- The SR architecture can be instantiated on various dataplanes
 - SR over MPLS (SR-MPLS)
 - SR over IPv6 (SRv6)
- Segment Routing can be directly applied to the MPLS architecture with no change on the forwarding plane
- Segment Routing can be applied to the IPv6 architecture with a new type of routing header called the SR header (SRH)

SR forwarding

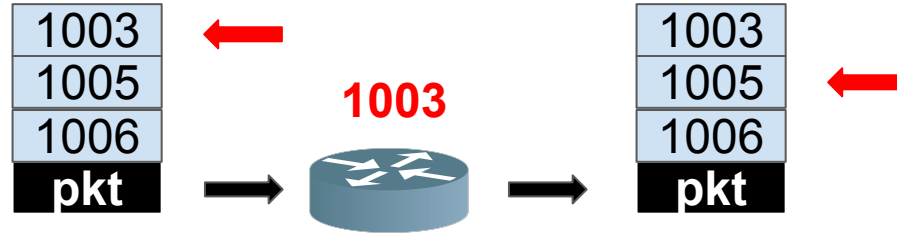
- SR nodes have a forwarding table that specifies the operation to perform on a received packet



- Three different operations:
 - PUSH**: the instruction consisting of the insertion of a segment at the top of the segment list

SR forwarding

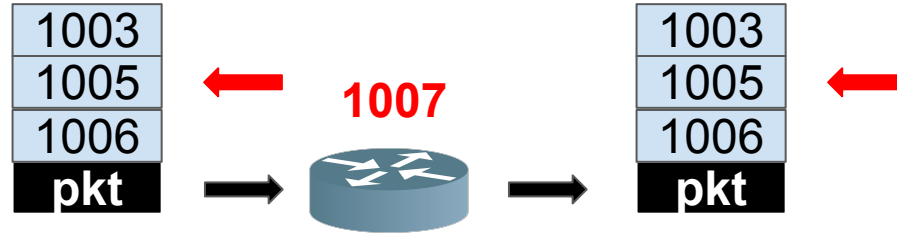
- SR nodes have a forwarding table that specifies the operation to perform on a received packet



- Three different operations:
 - PUSH**: the instruction consisting of the insertion of a segment at the top of the segment list
 - NEXT**: when the active segment is completed, NEXT is the instruction consisting of the inspection of the next segment

SR forwarding

- SR nodes have a forwarding table that specifies the operation to perform on a received packet



- Three different operations:
 - PUSH**: the instruction consisting of the insertion of a segment at the top of the segment list
 - NEXT**: when the active segment is completed, NEXT is the instruction consisting of the inspection of the next segment
 - CONTINUE**: the active segment is not completed and hence remains active

Outline

- Introduction to Segment Routing
- **Segment Routing Policy**
- SRv6 and Segment Routing extension Header
- Network Programming

SR Policy

- An **SR Policy** is identified through the tuple

<headend, color, endpoint>

- The **headend** is the node where the policy is instantiated/implemented
- The **endpoint** indicates the destination of the policy
 - headend and endpoint are specified as an IPv4 or IPv6 address
- The **color** is a 32-bit numerical value that associates the SR Policy with an intent (e.g., low-latency)

Candidate Path and Segment List

- An SR Policy is associated with one (or more) candidate path
- A **candidate path** is itself **associated with** a **Segment-List** (SID-List)
 - a SID-List represents a specific source-routed way to send traffic from the head-end to the endpoint of the corresponding SR policy
- A candidate path is either **dynamic** or **explicit**
- A headend may be informed about a candidate path for an SR Policy by various means including:
 - local configuration
 - PCE

SR Policy: summary

SR policy POL1 <headend, color, endpoint>

Candidate-path CP1

Preference 200

Weight W1, SID-List1 <SID11...SID1i>

Weight W2, SID-List2 <SID21...SID2j>

Candidate-path CP2

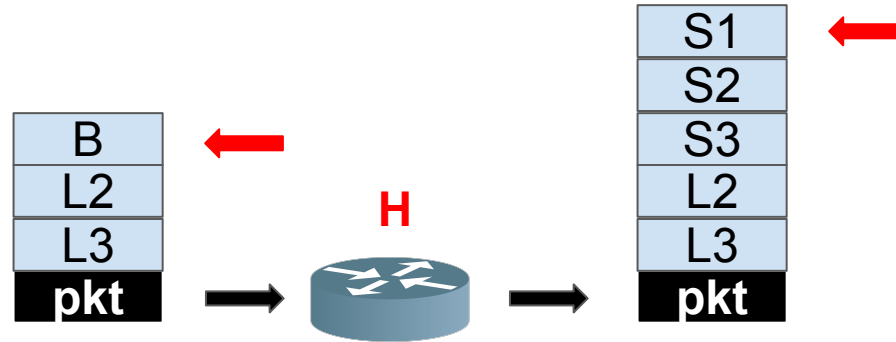
Preference 100

Weight W3, SID-List3 <SID31...SID3i>

Weight W4, SID-List4 <SID41...SID4j>

Binding SID

- An SR Policy installs a **BSID** entry in the forwarding plane with the action of **steering the packets matching this entry to the selected path**
- Let us assume that headend **H** has a valid SR Policy **P** of SID-List **<S1, S2, S3>** and BSID **B**



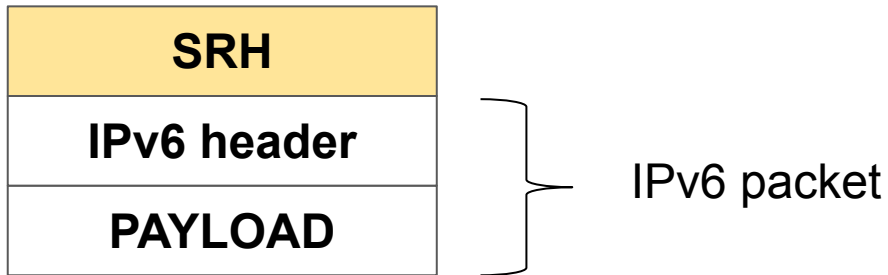
- **H** has steered the packet in the SR policy **P**

Outline

- Introduction to Segment Routing
- Segment Routing Policy
- SRv6 and Segment Routing extension Header
- Network Programming

SRH instantiation

- The source based routing model in case of SR over IPv6 dataplane (SRv6) is realized through the instantiation of the **Segment Routing Header (SRH)**
- The SRH is added to the packet by its source:
 - At the **node originating the packet** (host, server)
 - At the **ingress node** of an SR domain



Segment Routing Extension Header (SRH)

- A new type of the Routing Header is defined
 - the Segment Routing Header (SRH)

next header	hdr ext len	routing type	segments left
last entry	flags	tag	
segments list [0] (128 bit IPv6 address)			
.....			
segments list [n] (128 bit IPv6 address)			
optional type length value objects (variable)			

Segment Routing Extension Header (SRH)

- **Next Header:** Identifies the type of header immediately following the SRH
- **Hdr Ext Len:** length of the SRH header in 8-octet units
- **Routing Type:** TBD, to be assigned by IANA (suggested value: 4)
- **Segments Left:** it contains the index, in the Segment List, of the next segment to inspect
 - Segments Left is decremented at each segment
- **Last Entry:** contains the index, in the Segment List, of the last element of the Segment List
- **Flags:** 8 bits of flags
- **Tag:** tag a packet as part of a class or group of packets (packets sharing the same set of properties)
- **Segment List[n]:** 128 bit IPv6 addresses representing the nth segment in the Segment List
- **Type Length Value (TLV)**

SRH processing

- For the SRH holds the following property:
 - Only the router whose address is in the DA field of the packet header **MUST** inspect the SRH
- Segment Routing in IPv6 networks implies that the segment identifier is moved into the DA of the packet



- The DA of the packet changes at each segment termination/completion

My Local SID Table

- An SRv6-capable node N maintains a **MyLocalSID Table**
- This table contains all the **local SRv6 segments** explicitly instantiated at node N
- N is the parent node for these SID's
- Every SRv6 local SID instantiated has a specific instruction bounded to it

MyLocalSID Table

SID	instruction
...	...

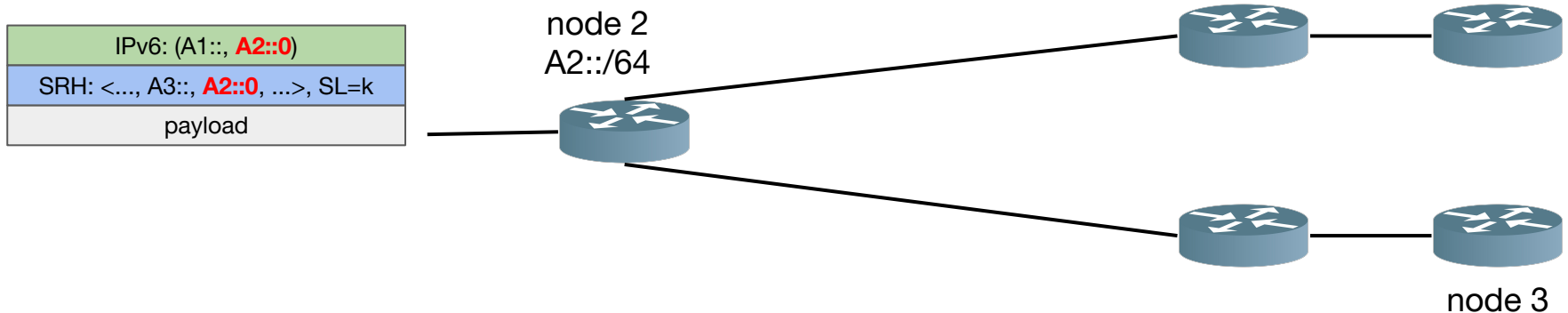
SRH Functions

- Segment Routing architecture defines a **segment as an instruction** or, more generally, a set of instructions (function)
- Two SRv6 basic functions are:
 - **End**
 - the endpoint (End) function is the base of the source routing paradigm
 - it consists of **updating the DA with the next segment** and forward the packet accordingly
 - **End.X**
 - the endpoint **layer-3 cross-connect function**

Endpoint Function (End)

- When a node receives a packet destined to DA "S" and:
 - S is an entry in the MyLocalSID table
 - the function associated with S is "End"

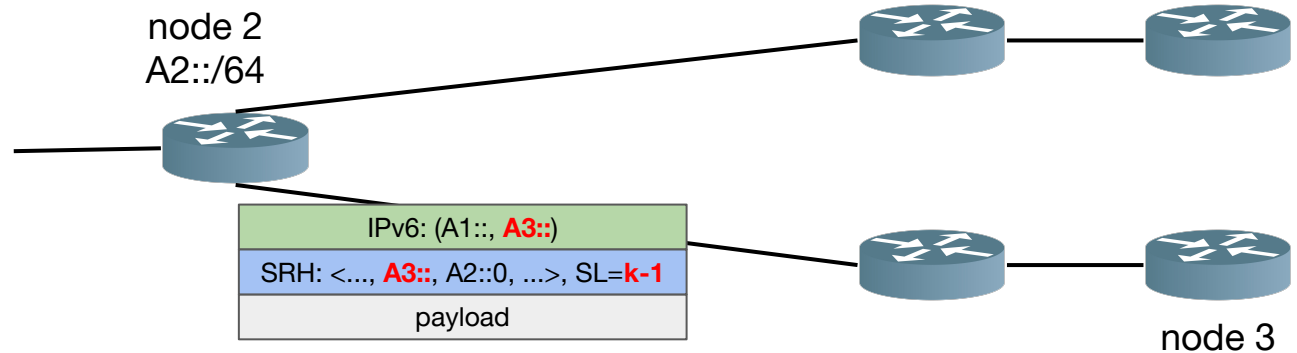
1. **IF** SegmentsLeft > 0 **THEN**
2. decrement SL
3. update the IPv6 DA with SRH[SL]
4. FIB lookup on updated DA
5. forward accordingly to the matched entry
6. **ELSE**
7. drop the SRH



Endpoint Function (End)

- When a node receives a packet destined to DA "S" and:
 - S is an entry in the MyLocalSID table
 - the function associated with S is "End"

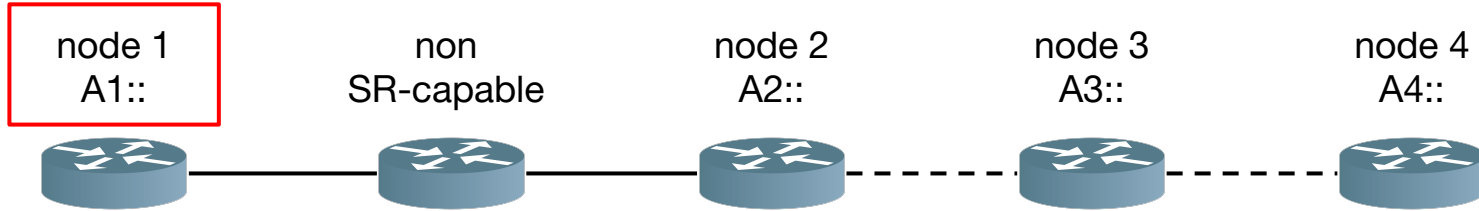
1. **IF** SegmentsLeft > 0 **THEN**
2. decrement SL
3. update the IPv6 DA with SRH[SL]
4. FIB lookup on updated DA
5. forward accordingly to the matched entry
6. **ELSE**
7. drop the SRH



Source Node

- Source Node is SR-capable
- SRH is created with:
 - Segment List in reversed order of the path
 - Segment List [0] is the **LAST** segment
 - Segment List [n-1] is the **FIRST** segment
 - Segment left is set to **n-1**
 - First segment is set to **n-1**
- IP DA is set to the first segment
- Packet is sent according to the IP DA
 - Normal IPv6 forwarding

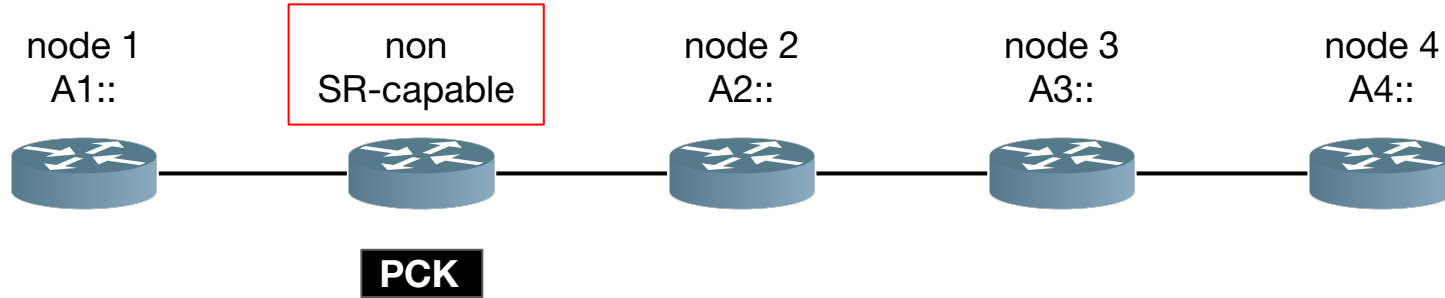
Source Node



PCK

payload length		next = 43	hop limit
source address = A1::			
destination address = A2::			
next header	len = 6	type = 4	SL = 2
First = 2	flags	tag	
segments list [0] = A4::			
segments list [1] = A3::			
segments list [2] = A2::			

Non-SR Transit Node

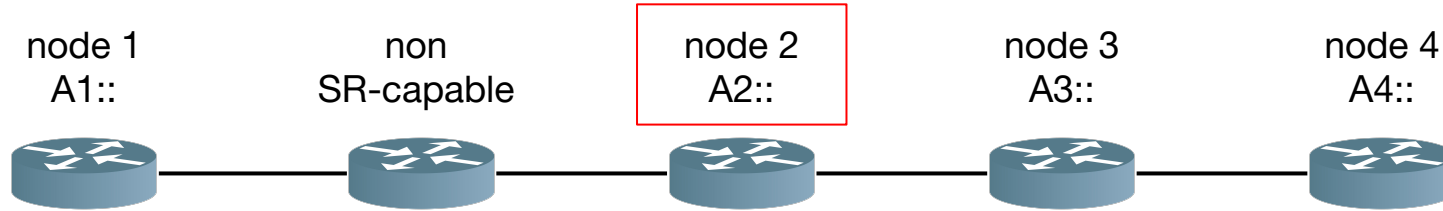


- Plain IPv6 forwarding
- **Solely based on IPv6 destination address**
- No SRH inspection or update

SR Segment Endpoints

- **SR Endpoints:** SR-capable nodes whose address is in the IP DA
- SR Endpoints inspect the SRH and do:
 - **IF Segments Left > 0, THEN**
 - Decrement Segments Left (-1)
 - Update DA with Segment List [Segments Left]
 - Forward according to the new IP DA

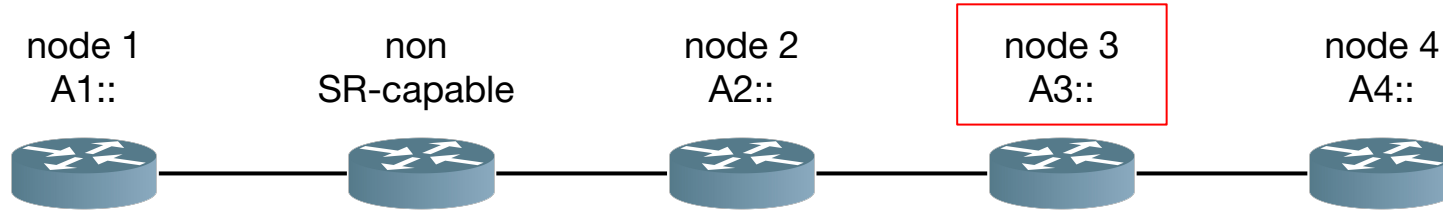
SR Segment Endpoints



PCK

vers	traffic class	flow label	
payload length		next = 43	hop limit
source address = A1::			
destination address = A3::			
next header	len = 6	type = 4	SL = 1
first = 2	flags	tag	
segments list [0] = A4::			
segments list [1] = A3::			
segments list [2] = A2::			
Payload			

SR Segment Endpoints



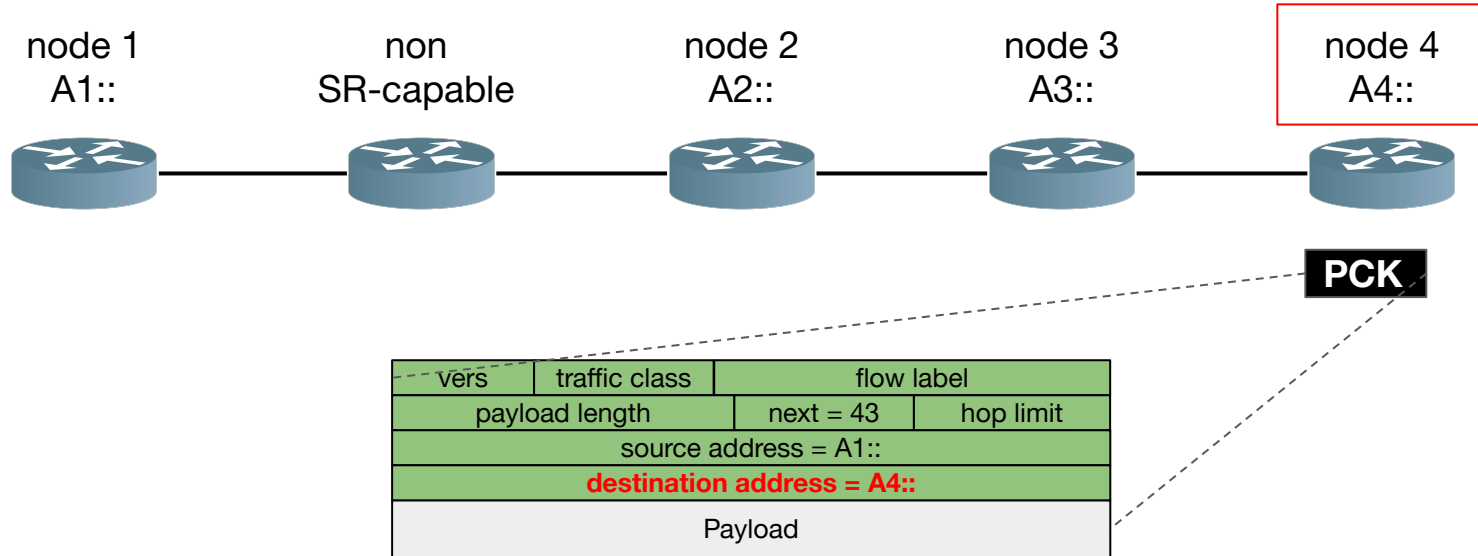
PCK

vers	traffic class	flow label	
payload length		next = 43	hop limit
source address = A1::			
destination address = A4::			
next header	len = 6	type = 4	SL = 0
first = 2	flags	tag	
segments list [0] = A4::			
segments list [1] = A3::			
segments list [2] = A2::			
Payload			

SR Segment Endpoints

- SR Endpoints: SR-capable nodes whose address is in the IP DA
- SR Endpoints inspect the SRH and do:
 - IF Segments Left > 0, THEN
 - Decrement Segments Left (-1)
 - Update DA with Segment List [Segments Left]
 - Forward according to the new IP DA
 - ELSE (Segments Left = 0)
 - Remove the IP and SR header
 - Process the payload

SR Segment Endpoints



Outline

- Introduction to Segment Routing
- Segment Routing Policy
- SRv6 and Segment Routing extension Header
- Network Programming

SRv6 Segment Format

<i>locator</i>	<i>function</i>	<i>argument</i>
1111:2222:3333:4444	5555:6666	7777:8888

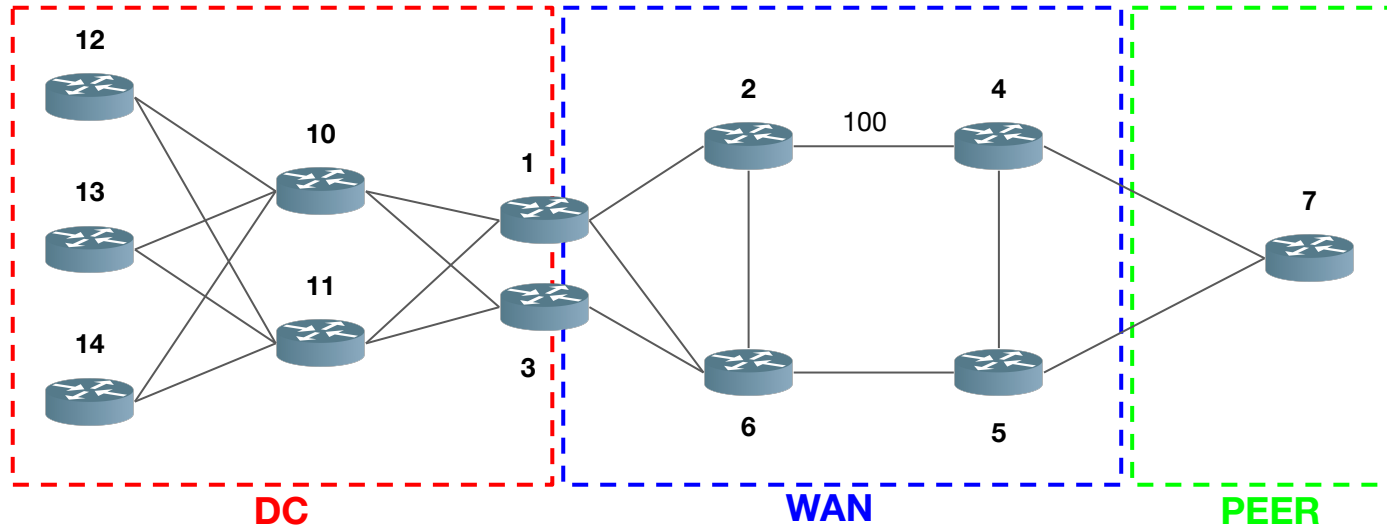
- An SRv6 local SID is logically represented as LOC:FUNCT
 - LOC is the L most significant bits
 - FUNCT is the 128-L least significant bits
- L is called the **locator** length and is flexible
- Most often the **LOC part of the SID is routable** and leads to the node which owns that SID
- The **FUNCT** part of the SID is an opaque identification of a **local function** bound to the SID
- A function may require additional arguments
 - the SRv6 Local SID will have the form LOC:FUNCT:ARGS

SRv6 Functions

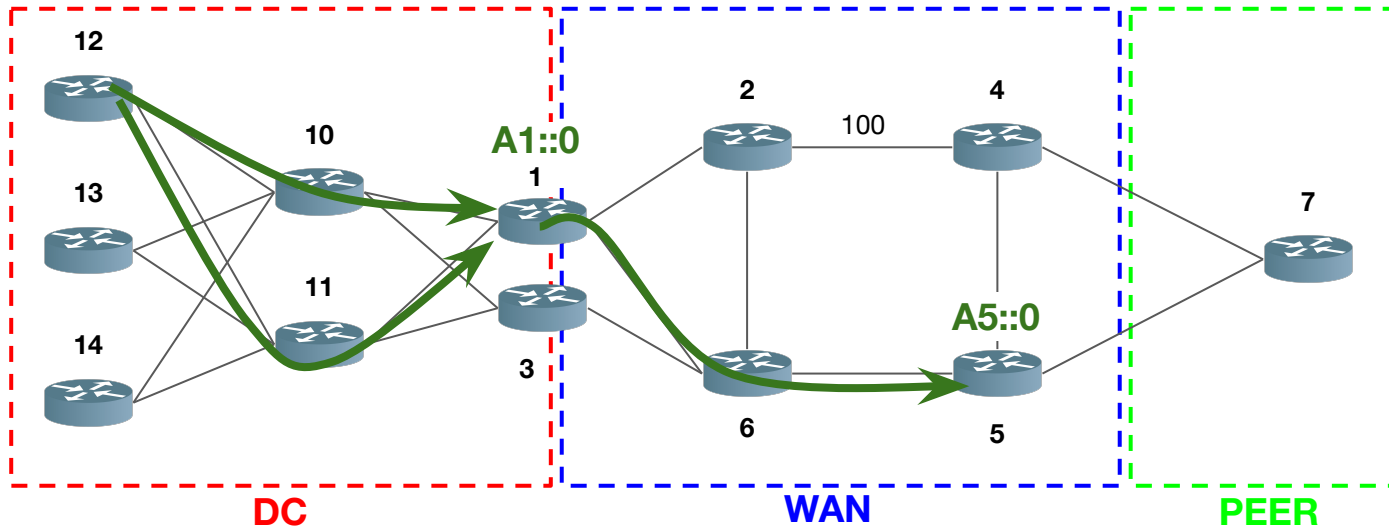
- Each entry of the MyLocalSID Table indicates the function associated with the local SID
- In practice, any function can be attached to a local SID
 - a node N can bind a SID to a local VM which can apply any complex function on the packet
- Some examples:
 - End
 - End.X
 - End.B6

SID allocation for illustration purpose

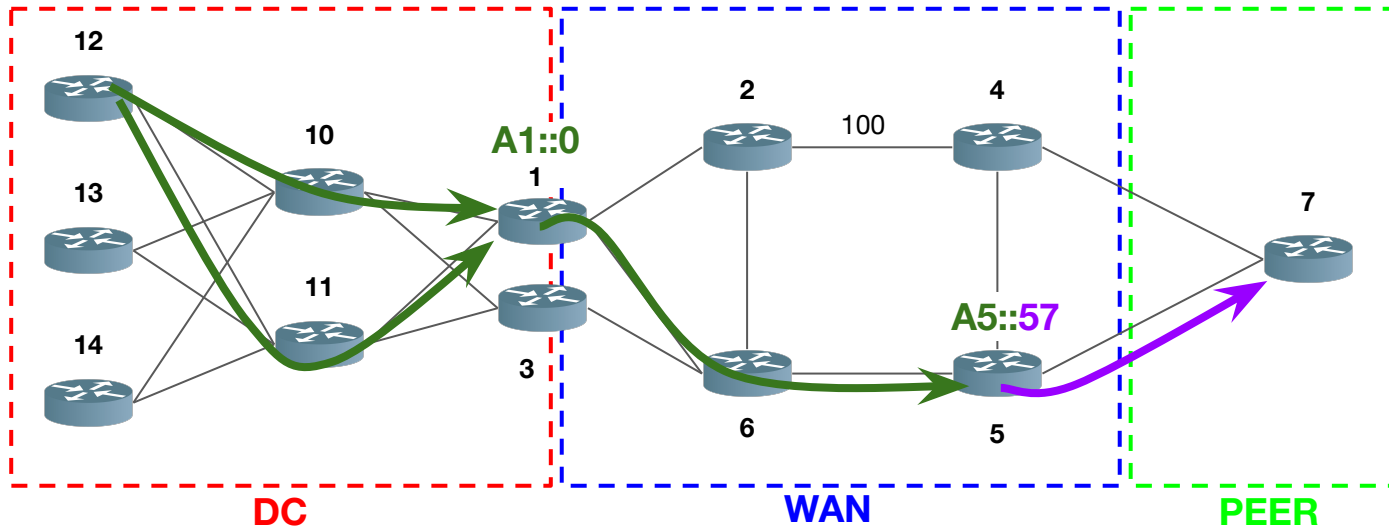
- Node **K** advertises prefix **AK::/64**
- The **function** is encoded in the **last 64 bits**
 - **0** denotes the **End** function
 - **CJ** denotes the **End.X** function on **link CJ**



A1::0 and then A5::0

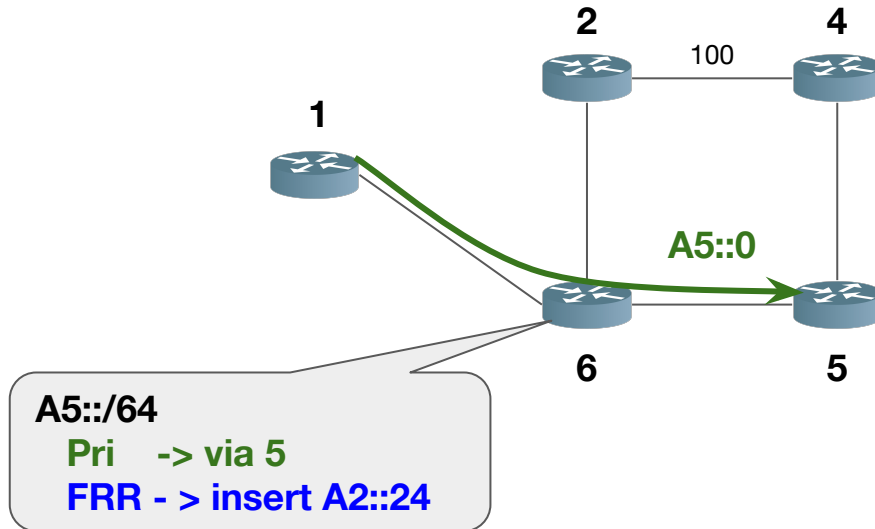


A1::0 and then A5::57



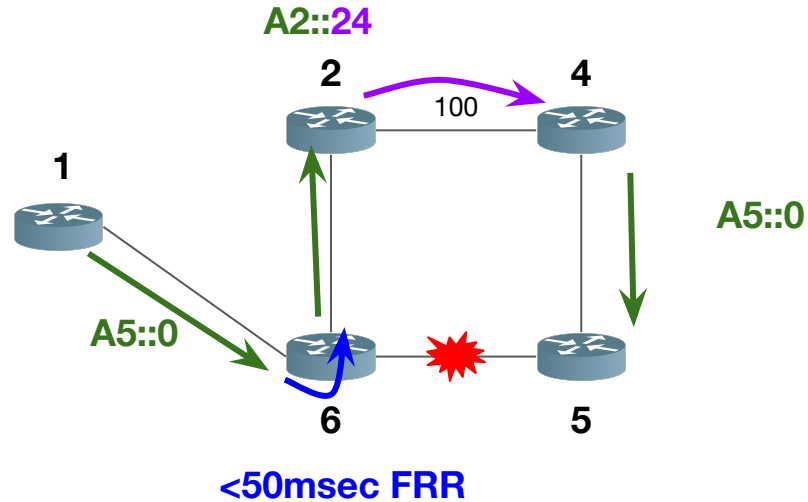
TILFA

- **50 msec protection** upon local link, node or SRLG failure

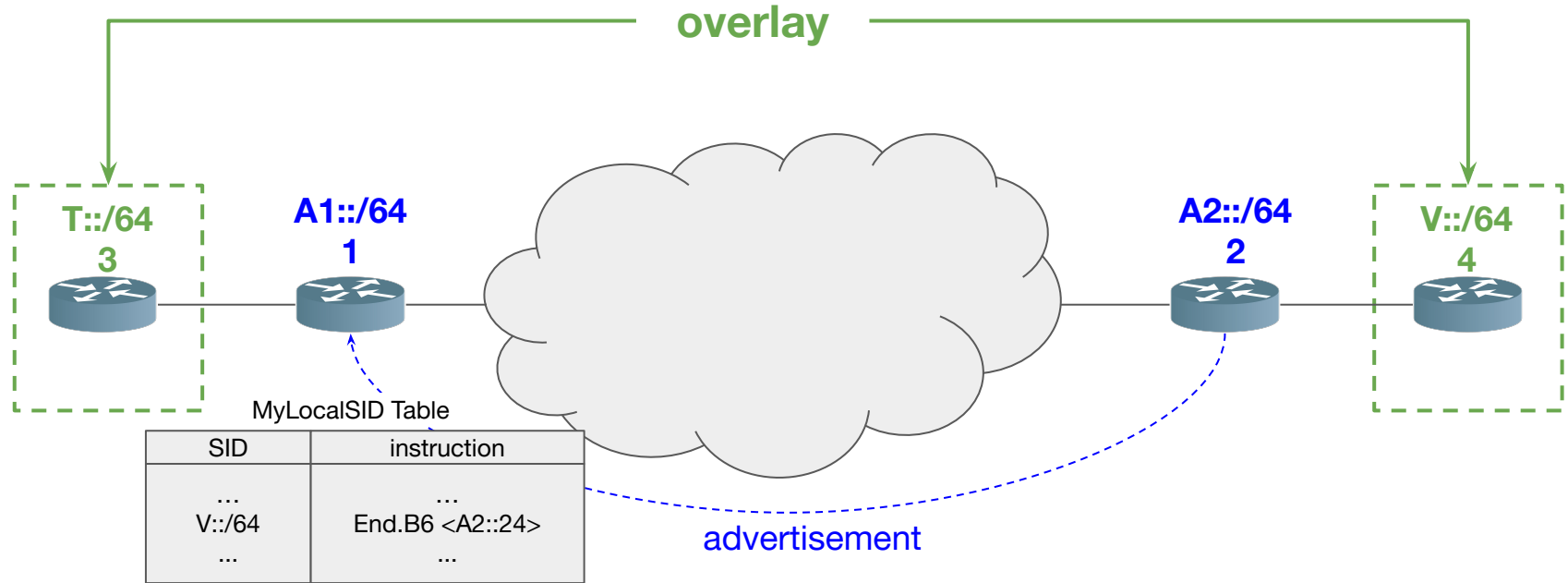


TILFA

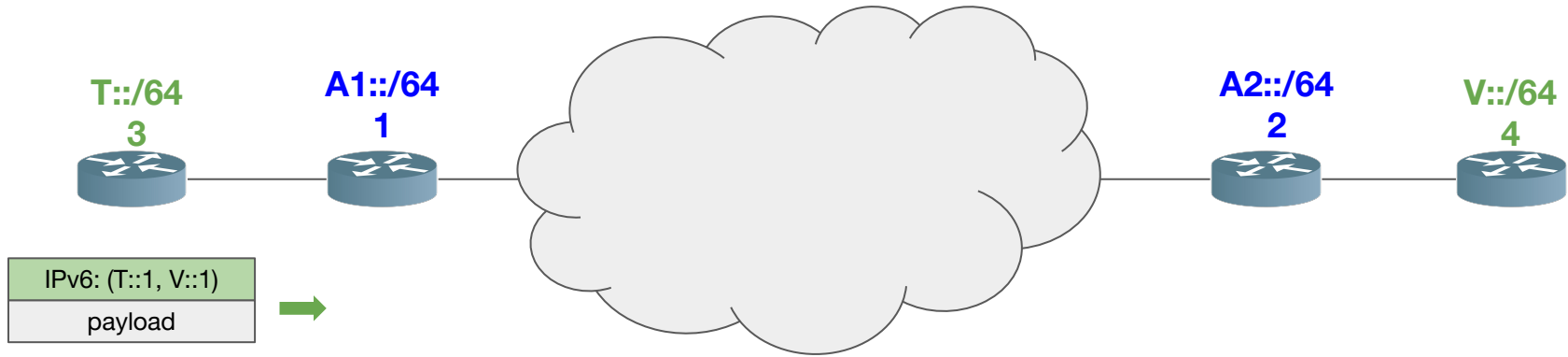
- 50 msec protection upon local link, node or SRLG failure



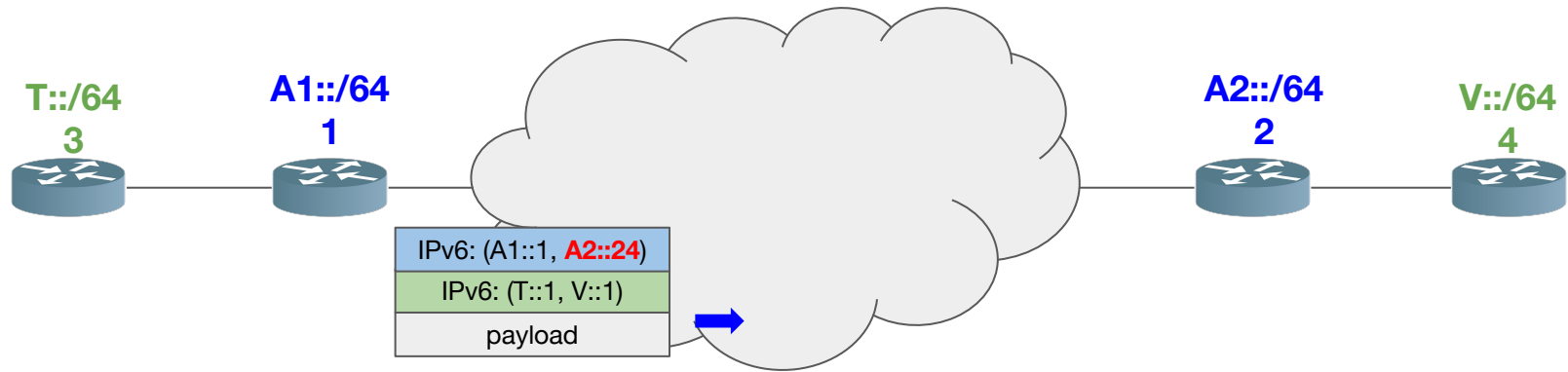
Overlay



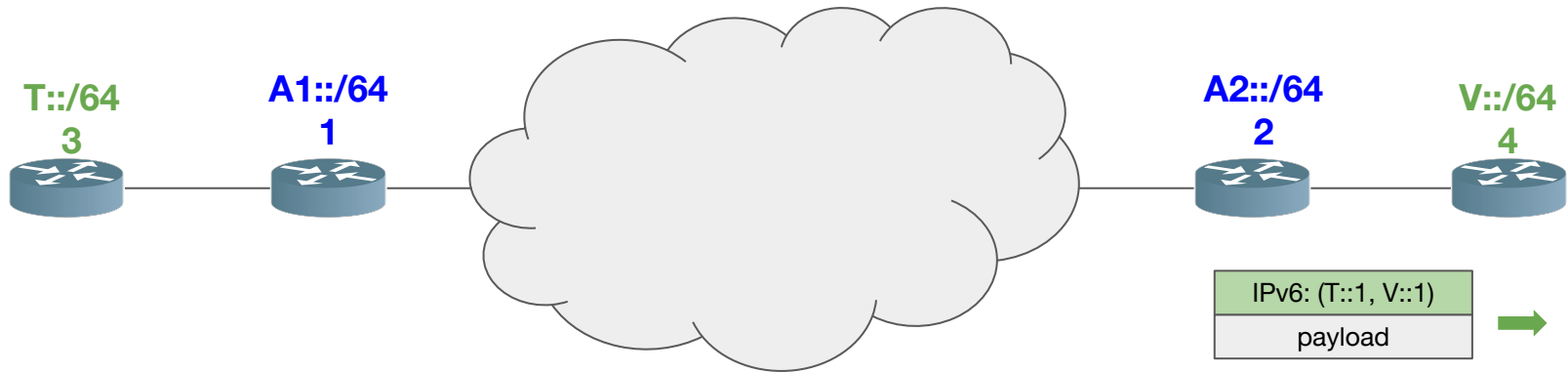
Overlay



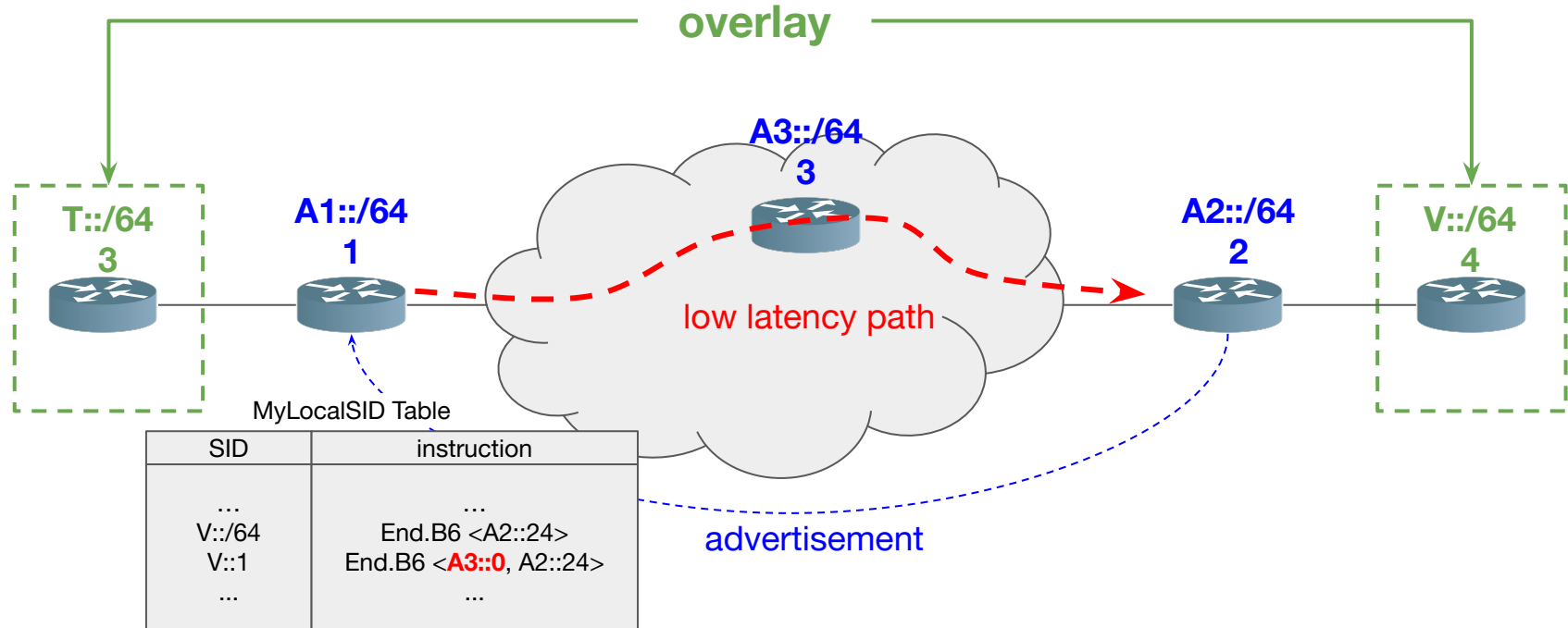
Overlay



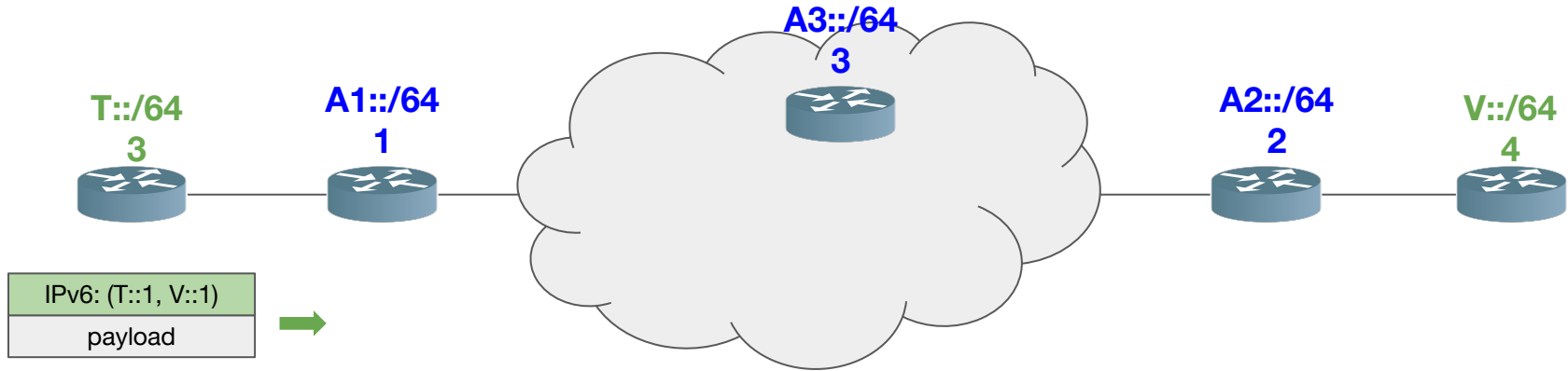
Overlay



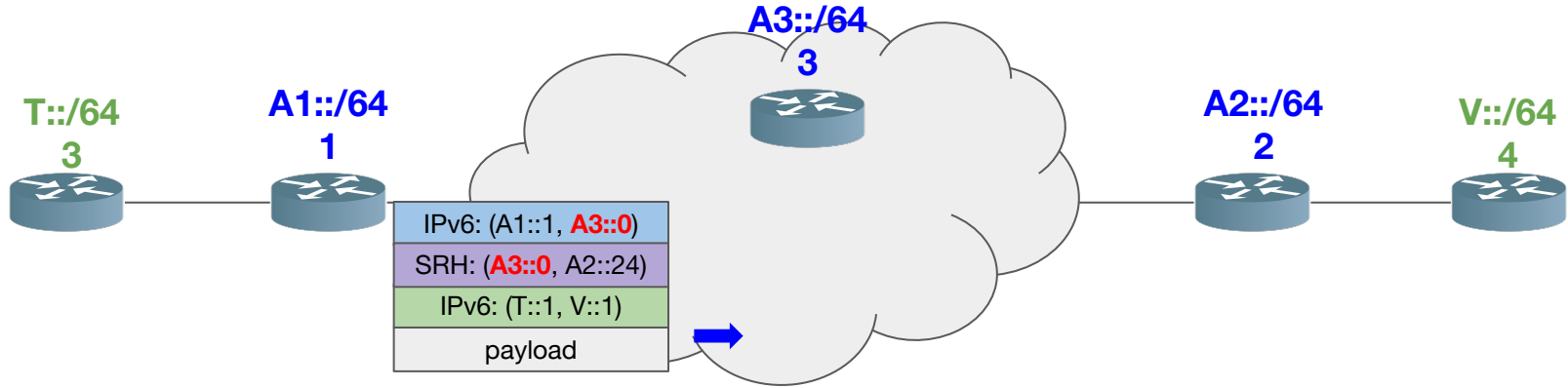
Overlay with underlay SLA



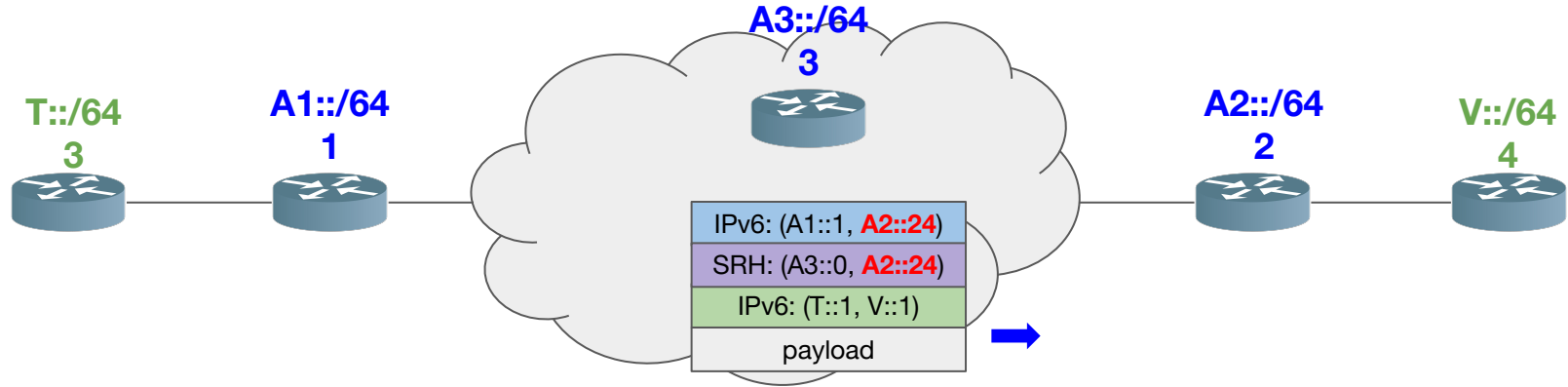
Overlay with underlay SLA



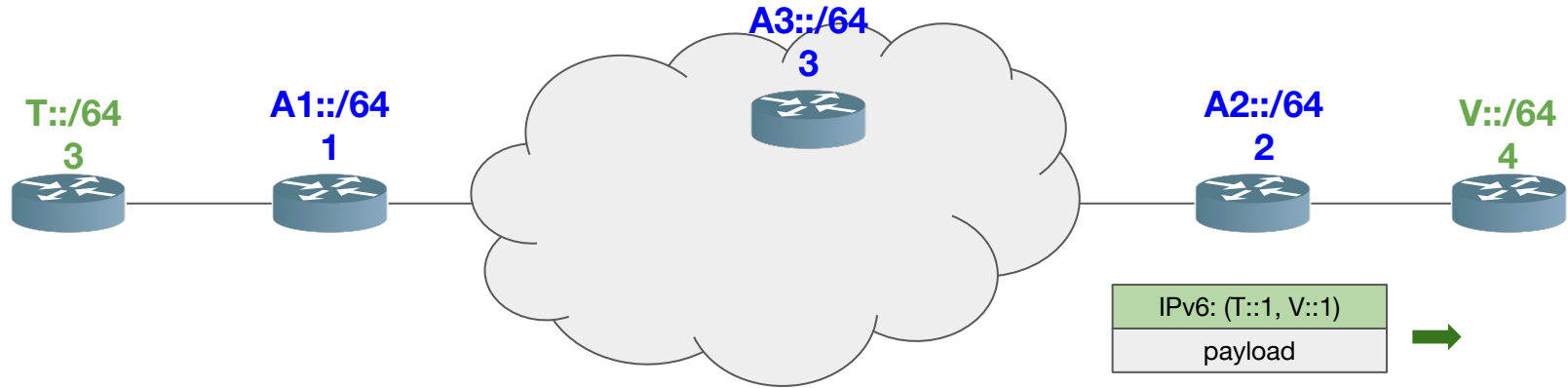
Overlay with underlay SLA



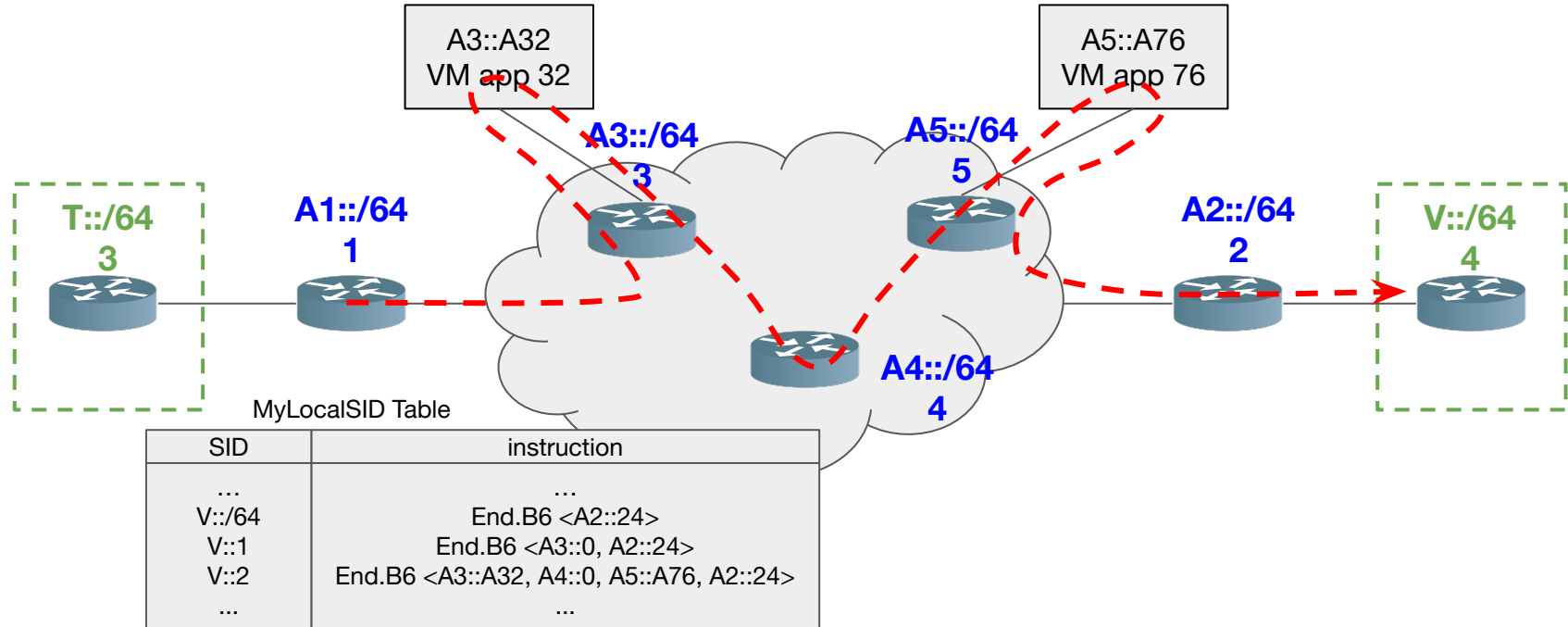
Overlay with underlay SLA



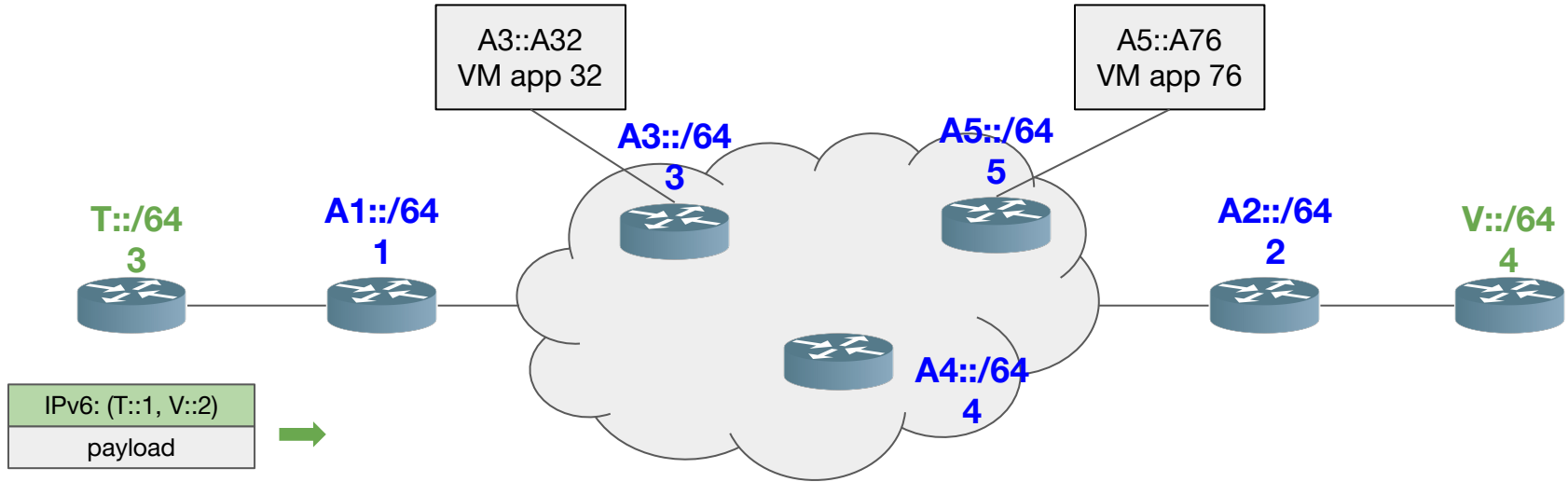
Overlay with underlay SLA



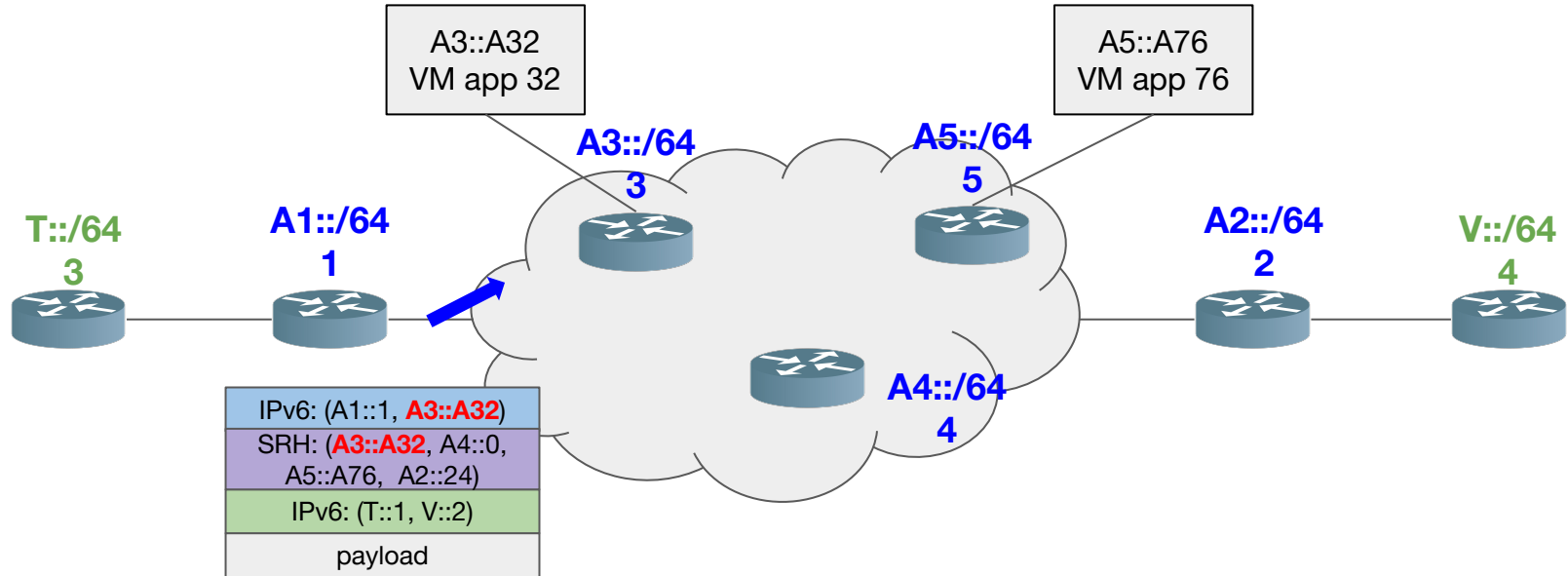
Integrated NFV



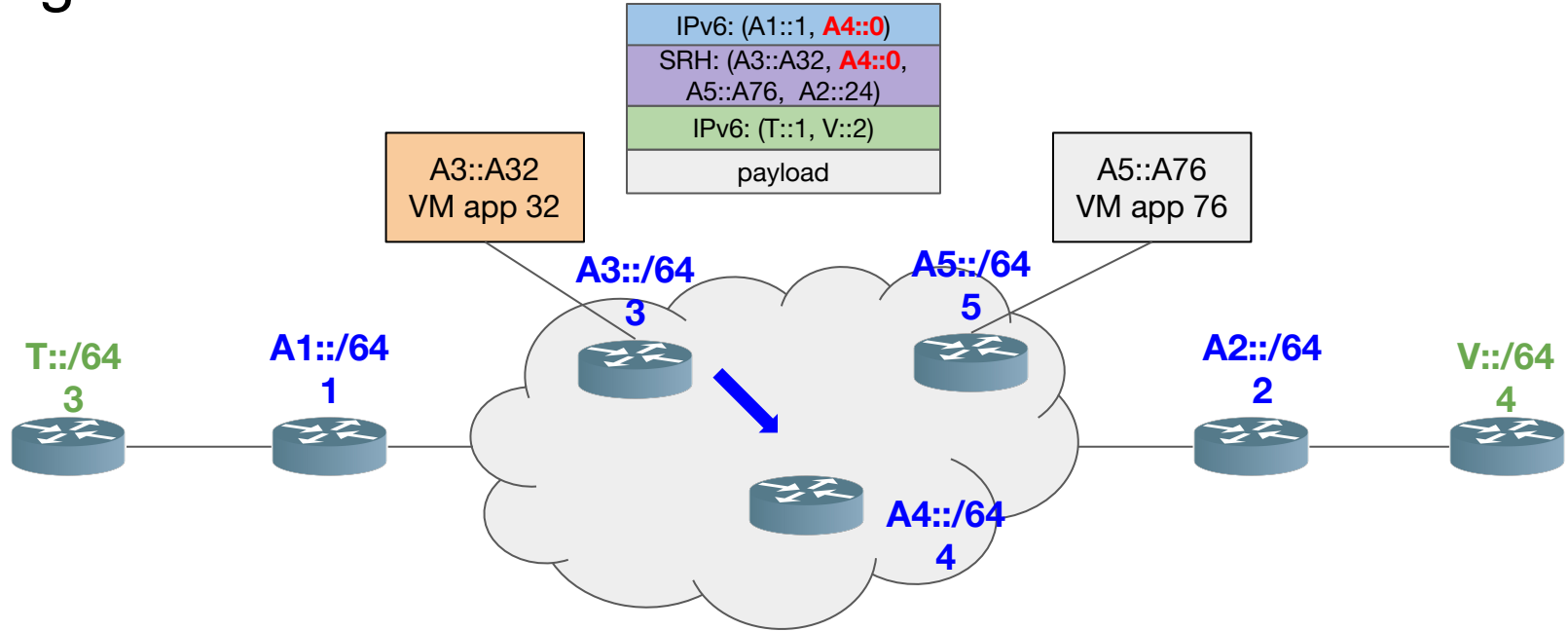
Integrated NFV



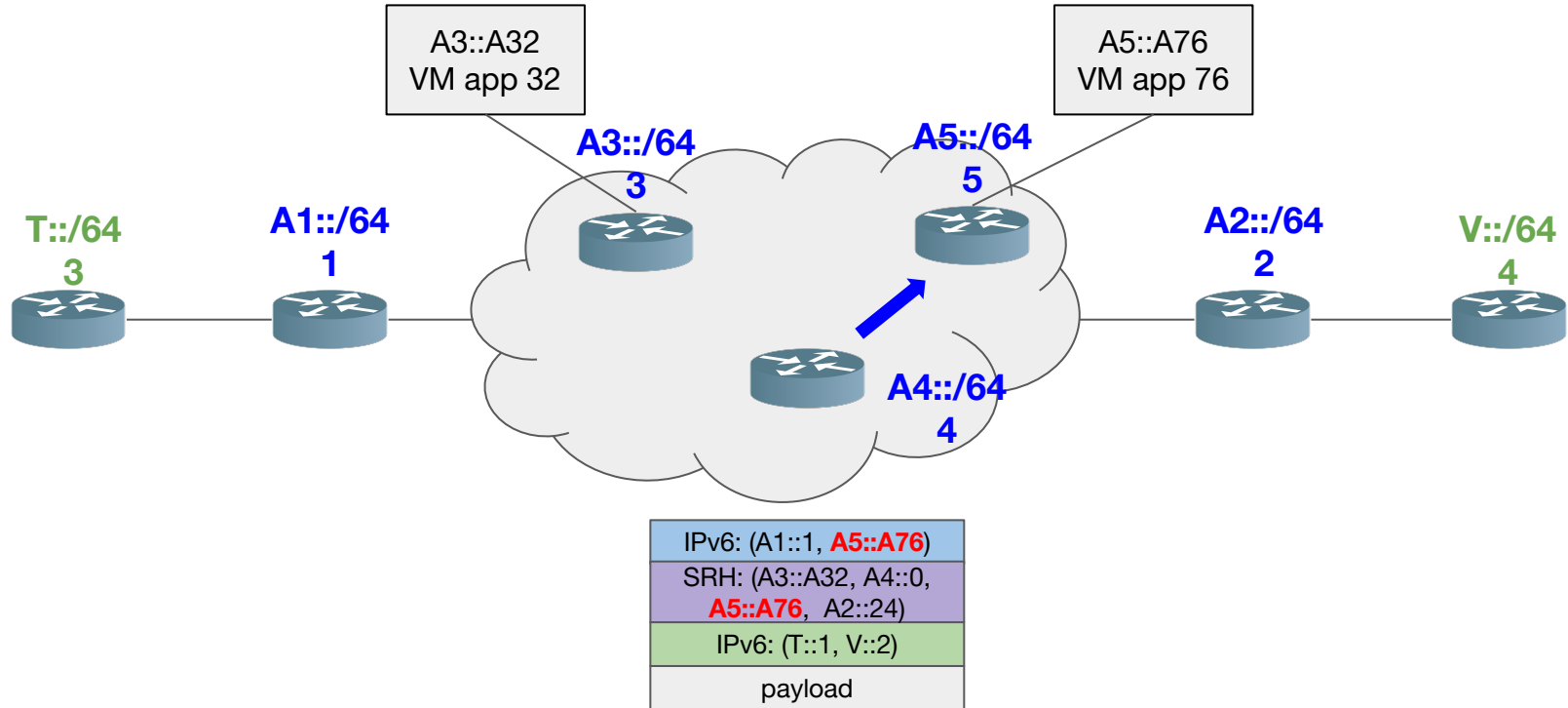
Integrated NFV



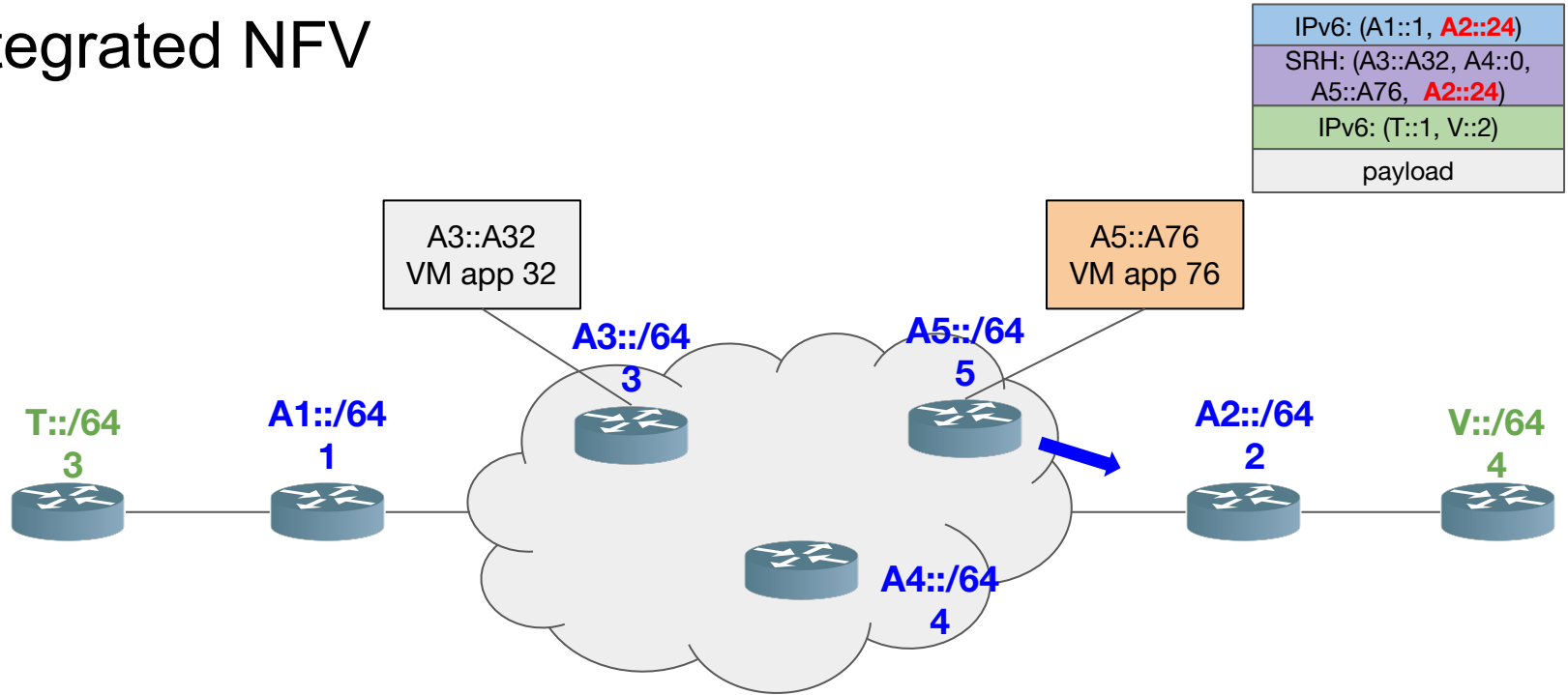
Integrated NFV



Integrated NFV



Integrated NFV



Integrated NFV

